

Befunge-93 Syntax Sheet

INTRODUCTION

Befunge93 is a programming language where code is written on a grid, not in straight lines. The program moves through this grid and reads instructions as it travels. It can go right, left, up, or down, so the path of execution is part of your program logic.

Think of it like a tiny robot walking over a grid of characters. Each character tells the robot what to do next.

Your code lives inside an 80×25 box, and every cell contains one character that usually acts as an instruction. **Writing outside this box will cause errors.**

Befunge93 mainly works using a stack. You push values onto it and pop values from it to do calculations, print output, and control flow.

They can also read and write the contents of any cell in the playfield, given its coordinates, thus Befunge93 code can be self-modifying. Characters are represented by their ASCII values within the stack, enabling operations like $A + D$, as they operate on the ASCII values of the characters.

BASIC SYNTAX

Befunge93 works using a stack. Almost every instruction either puts something on the stack or takes something from it.

Numbers	0 to 9	Push that number onto the stack. So 5 means “put 5 on the stack”.
Input	&	Takes an integer from the user and pushes it onto the stack.
	~	Takes a single character from the user and pushes its ASCII value onto the stack.
String Mode	“”	Turns string mode ON or OFF. While string mode is ON, every character you type is pushed onto the stack as its ASCII value until another " is found. This is mainly used to print text like "Hello".
Output	.	Pops the top value and prints it as a number.
	,	Pops the top value and prints it as a character.
Stack Operations	:	Duplicate the top value of the stack. If stack is empty, it treats the value as 0 and pushes two 0s.
	\	Swap the top two values of the stack. If there is only one value, it assumes the second value is 0.
	\$	Pop and discard the top value of the stack.
Terminate	@	Stops the program

DIRECTION POINTERS

In Befunge9393, the program doesn't just go left to right. The instruction pointer (think of it as a moving cursor) can change direction. Wherever it moves, it reads and executes the character in that cell.

By default, the program starts at the top-left corner and moves to the right.

>	Move right
<	Move left
^	Move up
v	Move down
?	Move in a random direction (up, down, left, or right).
#	Skip the next cell in the current direction.

If your code hits v, the program will start moving downward from that point.

You are not just writing instructions, you are drawing paths for the program to follow.

This is an infinite loop:

>v

^<

CONDITIONAL STATEMENTS

—	Horizontal IF: pop a value; set direction to right if value=0, set to left otherwise
	Vertical IF: pop a value; set direction to down if value=0, set to up otherwise

Trick 1: Using # before a condition

skips the next cell. You can use this to jump directly to a condition operator.

Examples:

```
#v_  
#^_
```

How it works:

- The instruction pointer reaches #
- It skips the arrow (v or ^)
- It lands on _
- _ checks the value and changes direction

This makes conditional logic shorter and cleaner.

You don't need long paths just to reach _ or |.

helps you "jump" straight to the condition.

MATHEMATICAL OPERATIONS

All math in Befunge9393 works using the stack.

It takes numbers from the stack, does the operation, and pushes the result back.

Addition	+	Pops b, then pops a, followed by pushing the result of $a + b$
Subtraction	-	Pops b, then pops a, followed by pushing the result of $a - b$
Multiplication	*	Pops b, then pops a, followed by pushing the result of $a * b$
Division	/	Pops b, then pops a, followed by then push the result of a/b , rounded down. b cannot be zero.
Modulus	%	Pops b, then pops a, followed by pushing the result of $a \% b$
Logical NOT	!	Pop a value. If the value is zero, push 1; otherwise, push zero
Greater Than	>	Pops b, then pops a, then push 1 if $a > b$, otherwise zero.

Important stack order rule:

The first popped value (b) is the second operand.

So, for subtraction and division it's always $a - b$ and a / b , not $b - a$.

SPECIAL INSTRUCTIONS

Apart from the above instructions, there exist 2 special instructions in Befunge93; `get(g)` and `put(p)`. These two are very powerful and must be used carefully. These are generally used to store data on the 2D playfield itself so that it becomes easier to access this data.

$g \rightarrow$ get from the grid	• Pops y, then x from the stack • Looks at position (x, y) in the program grid • Pushes the ASCII value stored there onto the stack
$p \rightarrow$ put into the grid	• Pops y, then x, then v • Writes the value v into position (x, y) of the grid

Trick 2: Using unused cells as memory (**p** and **g**)

Pick some cells that your code will never go through.
Use them to store values.

- $p \rightarrow$ store a value in a cell
- $g \rightarrow$ read the value from that cell

This works like variables in other languages.

It helps you save results and reuse values instead of keeping everything on the stack.

CODE EXAMPLES

1. Printing a number

```
23+.@
```

Push 2, push 3, add them → stack has 5 → . prints 5 → @ ends.

2. Printing a character

```
65, @
```

65 is ASCII for A.

, prints the character → output is A.

3. Printing a String

```
"!iH",,, @
```

- " starts string mode
- H and I go into the stack
- " ends string mode
- , prints I
- , prints H
- (Stack is LIFO, so order matters)

4. Using conditionals

```
0_@
```

Push 0 → _ sees 0 → goes right → hits @ → program ends. Change 0 to any non-zero number and it will go left instead.

5. Using get and put commands

```
" A" 01p 01g, @
```

- " A" pushes ASCII of space and A
- 01p stores A at grid position (0,1)
- 01g reads it back
- , prints it → outputs A

6. Example problem: Uppercase checker (Medium)

~:88*` \ 52\*9\*1+` ! * . @

Dry Run:

~	Inputs a character
:	Duplicates the top value on stack
88*	• Pushes 8 • Pushes 8, • * Computes the product and stores 64 on top of the stack
`	Checks if the input character is greater than 64 (ASCII Value of “@”)
\	Swaps the top value with the ASCII value of the input character
52*9*1+	• Pushes 5 • Pushes 2 • Computes their product and stores 10 on the top of the stack • Pushes 9 • Computes product of 10 and 9 • Stores 90 • Pushes 1 • Compute the sum of 90 and 1 and store 91
`	Checks if the input character is greater than 91 (ASCII value of “[“)
!	Store the complement of the value stored after comparison because the character should be within the range (64, 90) to be an Uppercase (check the ASCII table)
*	Computes the “AND” of input character being greater than 64 and input character being less than 90
.@	Prints the top value (0 if not uppercase and 1 if uppercase) and terminates the program

7. Example 2: Power Function (Hard)

109p>&19p>&:29p>0`#v_1.@

> >29g: 0` #v_09g.@

^p90*<g90<g91<p92<-1<

Dry Run:

109p	Stores 1 in the coordinate (0,9). This stores the result of the power function.
&19p	Stores the input value in the coordinate (1,9). This is the base of the power function
&:29p	Pushes the input value, duplicates it and stores it in the coordinate (2, 9). This is the exponent/power of the power function
0`#_1.@	Checks if the exponent is greater than zero. If 0, it prints 1 and terminates.
29g:0`#v_09g.@	Fetches the exponent from the coordinate (2, 9) and checks if greater than zero. If zero, the program prints the result in (0, 9) and terminates.
-1>29p	Decrements 1 from the exponent and stores the result in (2,9)
19g	Fetches the base from (1, 9).
09g	Fetches the current result (or product) from (0, 9)
*09p	Computes the product of the current result and the base. Stores the result again in (0, 9)

ASCII TABLE

ASCII	CHAR	ASCII	CHAR	ASCII	CHAR	ASCII	CHAR
0		32	[space]	64	@	96	`
1		33	!	65	A	97	a
2		34	“	66	B	98	b
3		35	#	67	C	99	c
4		36	\$	68	D	100	d
5		37	%	69	E	101	e
6		38	&	70	F	102	f
7		39	‘	71	G	103	g
8		40	(	72	H	104	h
9		41	)	73	I	105	i
10		42	*	74	J	106	j
11		43	+	75	K	107	k
12		44	,	76	L	108	l
13		45	-	77	M	109	m
14		46	.	78	N	110	n
15		47	/	79	O	111	o
16		48	0	80	P	112	p
17		49	1	81	Q	113	q
18		50	2	82	R	114	r
19		51	3	83	S	115	s
20		52	4	84	T	116	t
21		53	5	85	U	117	u
22		54	6	86	V	118	v
23		55	7	87	W	119	w
24		56	8	88	X	120	x
25		57	9	89	Y	121	y
26		58	:	90	Z	122	z
27		59	;	91	[	123	{
28		60	<	92	\	124	
29		61	=	93	]	125	}
30		62	>	94	^	126	~
31		63	?	95	_	127	

